Large Language Models in Data Science

Week 4: Text Classification — Compare Before Choosing

Sebastian Mueller
Aix-Marseille Université
2026–2027

Which Method Would You Put in the Workflow?

Week 3: generate an analysis, run it and check the result.

Today: several methods can perform the same task. How do we choose?

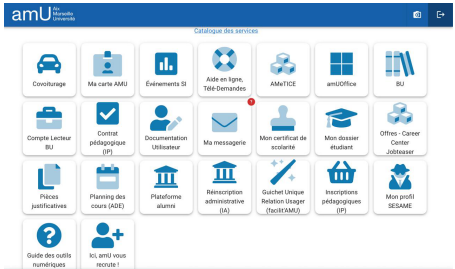
Our task

A student asks for help. Select the university service that should receive the request.

We will compare four approaches: regex rules, embeddings with logistic regression, a pretrained classifier using intent descriptions and a prompted LLM.

The decision needs evidence: correct labels, types of errors, response time and cost.

From a Student's Request to a Service



AMU portal: the setting for our teaching example.

Student's request
"Ouvrir ma messagerie"

↓

Classifier
Predicts a label for the request

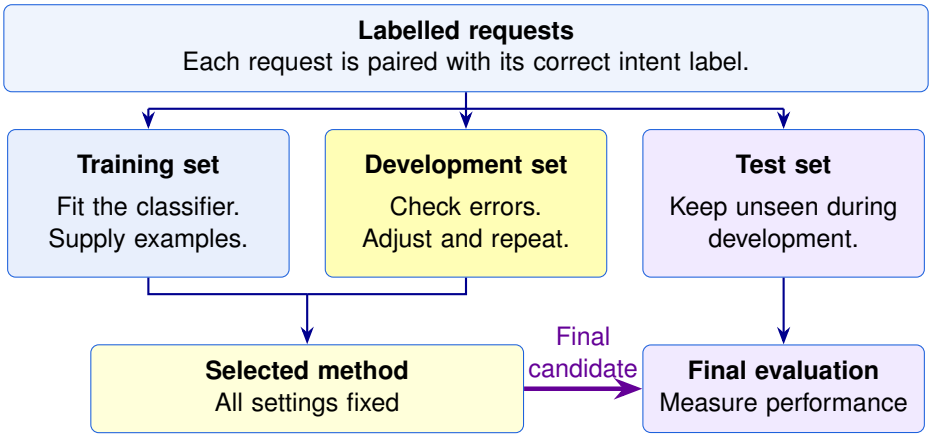
↓

`check_email`
Route to the email service

Define the Labels Before Building a Classifier

Label	Intent	Example request
`get_schedule`	View class timetable	“Voir mes cours de demain”
`check_email`	Read student email	“Ouvrir ma messagerie”
`register_classes`	Register for classes	“Je dois faire mon IP”
`get_student_card`	Manage student card	“J’ai perdu ma carte”
`find_library_info`	Find library information	“La BU est ouverte ?”

Split the Labelled Data into Three Sets



Four approaches for classification

Method	How the label is chosen	What we supply
Regex rules	Match specified text patterns.	Rules written by us.
Embeddings + logistic regression	Convert the request to a vector, then classify that vector.	Labelled training requests.
Pretrained classifier	Give it the request and five possible intents. It scores each intent; select the highest.	Intent descriptions, such as “student email”.
Prompted LLM	Ask the model to classify the request and return its label.	Label definitions and a reply format.

1. Regex Rules: Match Text Patterns

A regular expression (regex) specifies a text pattern. Here, | means “or”: match `messagerie` or `email`.

```python
import re

def email_rule(query):
    if re.search("messagerie|email", query.lower()):
        return "check_email"
    return None  # No matching rule; no label selected.

email_rule("Ouvrir ma messagerie")  # "check_email"
```

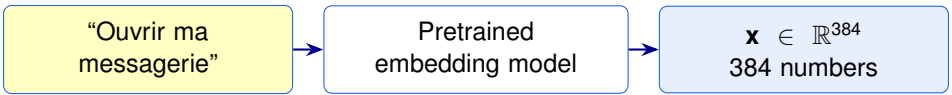
The complete classifier has rules for all five labels and an explicit policy when several rules match or none does.

A request the rule misses

“Consulter mes e-mails” returns `None`: the pattern `email` does not match `e-mail`.

2. Embeddings: Represent the Whole Request by a Vector

In Week 1, vectors represented text. Here we use a pretrained **sentence-embedding model**: it converts a request into one fixed-length vector.



In the lab we use multilingual MiniLM. It supports French and produces 384 numbers per request.

The vector has no intent label yet. We must learn a mapping from these features to the five labels.

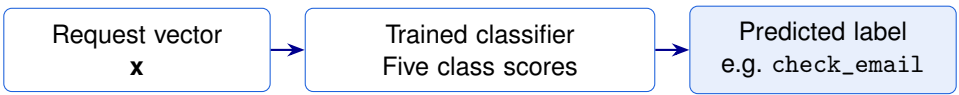
Train a Classifier on Those Vectors

Pair each training request's vector with its reference label. Fit a **multinomial logistic-regression classifier** to predict those labels.

For a new vector $\mathbf{x}$, calculate one score for each class c:

$$s_c = \mathbf{w}_c^\top \mathbf{x} + b_c, \qquad \widehat{y} = \operatorname*{argmax}_c s_c.$$

$\mathbf{x}$ is the request's vector; $\mathbf{w}_c$ is the class's weight vector; b_c is its bias. Training learns the weights and biases. $\widehat{y}$ is the predicted label: the class with the largest score.



We keep the embedding model fixed.

3. Pretrained Classifier

We supply the request and a description of each of our five labels.

Our label	Description supplied to the model
`get_schedule`	View class timetable
`check_email`	Read student email
`register_classes`	Register for classes
`get_student_card`	Manage student card
`find_library_info`	Find library information

Our model is multilingual mDeBERTa, already trained to judge whether a text supports a proposed statement.

For example: does “Ouvrir ma messagerie” support “The student wants to read student email”?

The model scores this support for each intent. Select the label with the highest score.

4. Prompt an LLM to Return a Label

Reuse Week 3's API call: system instructions define the task; the user message supplies the request.

```
System instructions: Classify the student's intent.
get_schedule: class timetable and teaching rooms
check_email: student email and inbox
register_classes: course registration
get_student_card: student card
find_library_info: library hours, books and borrowing
Return JSON: one "label" field with a label above.

User message: Ouvrir ma messagerie
```

Desired reply for this example:

```
{"label": "check_email"}
```

Here the model generates the JSON reply as text. We parse its label and compare it with the reference label.

A Valid Reply Can Still Select the Wrong Service

Keep the two checks from Week 3:

Reply	**Format and allowed label**	**Correct label?**
`{"label": "check_email"}`	Valid	Yes, for our request.
`{"label": "get_schedule"}`	Valid	No.
`The student wants email.`	Invalid: not the required JSON.	No usable label returned.
No API response	Request failed.	No usable label returned.

Structured output can constrain the format and allowed labels where supported. It does not guarantee the correct label.

Accuracy: How Many Requests Were Classified Correctly?

$$\text{Accuracy} = \frac{\text{number of correct labels}}{\text{number of test requests}}.$$

Illustrative counts, not model results

Correct label	43
Wrong allowed label	5
Invalid reply or no match	1
API failure	1
Total test requests	50

In this example, accuracy is $43/50 = 86\%$.

The two requests without usable labels stay in the denominator. We also report their failure types separately.

If retries are allowed, fix the policy before testing and include their time and API usage.

The Confusion Matrix Shows Which Labels Are Confused

A **confusion matrix** counts reference labels against predicted labels. Rows below are the reference labels; columns are the predicted labels.

Expected \ predicted	Schedule	Email	Register	Card	Library	No label
Schedule	8	0	1	0	0	1
Email	0	9	0	1	0	0
Register	0	0	9	0	0	1
Card	0	0	0	9	1	0
Library	1	0	0	1	8	0

Same illustrative 50 requests: the diagonal contains 43 correct labels. “No label” combines the invalid reply and API failure.

Which errors send students to the wrong service?

Precision and Recall Answer Different Questions

Focus on the **library** class in the same matrix. There are 10 actual library requests; 8 are classified correctly. One card request is also classified as library.

$$\text{Library recall} = \frac{\text{correct library predictions}}{\text{all actual library requests}} = \frac{8}{10} = 80\%.$$

Of the students who need the library, how many do we send there?

$$\text{Library precision} = \frac{\text{correct library predictions}}{\text{all library predictions}} = \frac{8}{9} \approx 89\%.$$

Of the students sent to the library, how many actually need it?

Report these measures for every class. Overall accuracy can hide poor performance on a less frequent service.

Measure Prediction Time and Token Usage

- **Prediction time**: how long it takes to obtain a label for a request. Start with the model already loaded; exclude training time.
- **API token usage**: record the input and output tokens reported by the API for each request.

Improve a Regex Rule with an LLM

Our rule misses “Consulter mes e-mails”. Its reference label is `check_email`.

Ask the LLM to extend the rule

Our Python regex is `messagerie|email`. It misses “Consulter mes e-mails”. Modify it to match both “email” and “e-mail”, while keeping “messagerie”.

A possible revision:

```
pattern = "messagerie|e-?mail"
re.search(pattern, "Consulter mes e-mails".lower())
```

`-?` means the hyphen is optional: both spellings now match.

Run the revised classifier on all development requests. Count corrected predictions and any new errors before keeping the change.

Generate Synthetic Debugging Data with an LLM

Synthetic data are invented examples. Ask an LLM to create ten requests for our development set:

```
Create 10 French student requests, two per label:
get_schedule: view class timetable
check_email: read student email
register_classes: register for classes
get_student_card: manage student card
find_library_info: find library information
Vary the wording; include abbreviations and typos.
Each request must express one intent.
Return CSV with columns text,label.
```

1. Read each request and check its assigned label.
2. Run the classifier on these requests and inspect the mistakes.
3. Revise the rules or prompt, then rerun the development set.

Next Week: From a Label to an Answer

“J’ai perdu ma carte étudiante. Comment la remplacer ?”

This week: classify the request

Predict the label `get_student_card`.

Next week: answer the question using university documents

1. Find the document describing how to replace a student card.
2. Give the relevant passage to the LLM with the question. This passage is the added context.
3. Ask the LLM to explain the procedure using that passage.

If the document lacks the replacement procedure, the LLM can request another search, inspect the result and decide whether it can answer.

Lab: Compare the Four Methods and Defend a Choice

1. Inspect the requests and agree what each of the five labels means.
2. Prepare separate training, development and final test examples.
3. Run the four classifiers. Inspect at least one complete prediction from each method.
4. Improve one method using development errors, then freeze its settings.
5. Compare the final methods on the same test requests: correctness, per-class errors, failures, latency and API usage.

Decision: which method would you deploy for this task, and what evidence or additional data would you need before doing so?

Further Reading: Classification and Evaluation

- scikit-learn: common pitfalls and data leakage — why model choices must not use the final test examples.
- scikit-learn: classification metrics — accuracy, confusion matrices, precision and recall.
- Multilingual MiniLM model card — the fixed sentence-embedding model used before logistic regression.
- Hugging Face: zero-shot classification pipeline and the multilingual NLI model card — how candidate descriptions become text pairs.